

NORMALIZACIÓN DE BASES DE DATOS

Cómo pasar de una tabla llena de datos repetidos y mezclados a un esquema limpio, sin redundancia y sin anomalías, en tres pasos: 1FN, 2FN y 3FN.

Estudiante: Silverio Villeda Barradas

Grupo: 3DSM

Tema: Normalización de bases de datos

Normalizar es reorganizar las tablas de una base de datos para que cada dato viva en un solo lugar. El objetivo no es "tener más tablas", sino evitar tres problemas concretos: **redundancia** (repetir el mismo dato en muchas filas), **anomalías de actualización** (cambiar un dato y olvidar actualizarlo en otra fila) y **anomalías de borrado** (perder información útil al eliminar una fila). Cada Forma Normal (FN) corrige un tipo específico de problema, apoyándose siempre en la anterior.



1FN PRIMERA FORMA NORMAL

Una tabla está en **1FN** cuando cada columna guarda **un solo valor atómico** (nada de listas dentro de una celda), no existen columnas repetidas para el mismo dato, y hay una **llave primaria** que identifica cada fila de forma única.

× Pedidos (sin normalizar)		
id_pedido	cliente	productos
101	Ana Ruiz	Café, Pan, Yogur
102	Luis Pérez	Torta

✓ Pedidos (1FN)		
id_pedido	cliente	producto
101	Ana Ruiz	Café
101	Ana Ruiz	Pan
101	Ana Ruiz	Yogur
102	Luis Pérez	Torta

EJEMPLO APLICADO — SQL

```
-- Cada fila = un producto de un pedido, valores atómicos
CREATE TABLE pedidos_1fn (
  id_pedido INT,
  cliente VARCHAR(60),
  producto VARCHAR(40),
  PRIMARY KEY (id_pedido, producto)
);
```

2FN SEGUNDA FORMA NORMAL

Una tabla está en **2FN** cuando ya cumple 1FN **y** además, si su llave primaria está formada por varias columnas, **ningún dato depende de solo una parte** de esa llave. Si algo depende de una sola parte, se mueve a su propia tabla.

× Detalle_Pedido (viola 2FN)			
id_pedido·PK	id_prod·PK	cantidad	nombre_prod
101	P01	2	Café
102	P01	1	Café

✓ Detalle_Pedido (2FN)		
id_pedido·PK	id_prod·PK/FK	cantidad
101	P01	2

✓ Productos (2FN)	
id_prod·PK	nombre_prod
P01	Café

Por qué nombre_prod solo depende de id_prod, no del pedido completo — por eso se repite y puede quedar inconsistente si un producto cambia de nombre.

EJEMPLO APLICADO — SQL

```
CREATE TABLE productos (
  id_prod VARCHAR(5) PRIMARY KEY,
  nombre_prod VARCHAR(40)
);
CREATE TABLE detalle_pedido (
  id_pedido INT, id_prod VARCHAR(5), cantidad INT,
  PRIMARY KEY (id_pedido, id_prod),
  FOREIGN KEY (id_prod) REFERENCES productos(id_prod)
);
```

3FN TERCERA FORMA NORMAL

Una tabla está en **3FN** cuando cumple 2FN **y** ningún dato que no sea llave depende de **otro dato que tampoco es llave** (dependencia transitiva). Si un dato depende de otro dato "normal", ambos se separan.

× Clientes (viola 3FN)			
id_cliente·PK	nombre	id_ciudad	nombre_ciudad
1	Ana Ruiz	C10	Tula de Allende

✓ Clientes (3FN)		
id_cliente·PK	nombre	id_ciudad·FK
1	Ana Ruiz	C10

✓ Ciudades (3FN)	
id_ciudad·PK	nombre_ciudad
C10	Tula de Allende

Cadena id_cliente → id_ciudad → nombre_ciudad: nombre_ciudad depende de id_ciudad, que a su vez depende de id_cliente. Esa dependencia "en cadena" es la transitiva que 3FN elimina.

EJEMPLO APLICADO — SQL

```
CREATE TABLE ciudades (
  id_ciudad VARCHAR(5) PRIMARY KEY, nombre_ciudad VARCHAR(50)
);
CREATE TABLE clientes (
  id_cliente INT PRIMARY KEY, nombre VARCHAR(60), id_ciudad VARCHAR(5),
  FOREIGN KEY (id_ciudad) REFERENCES ciudades(id_ciudad)
);
```

CONCLUSIÓN

Normalizar hasta 3FN evita que un mismo dato viva repetido en decenas de filas y, con eso, evita que se vuelva inconsistente. El costo es tener más tablas y más JOINs al consultar — un balance que casi siempre vale la pena, y que en sistemas de altísimo tráfico a veces se relaja a propósito (desnormalización controlada) para ganar velocidad de lectura.

FUENTES CONSULTADAS

- Elmasri, R. & Navathe, S. — Fundamentals of Database Systems
- Oracle Database Concepts — Data Normalization
- PostgreSQL Documentation — Chapter on Schema Design
- GeeksforGeeks — Normal Forms in DBMS